

Descriptor: *AEGIS Threat Matrix for Agentic AI Systems (ATX-1)*

KENNETH TANNENBAUM¹ (MEMBER, IEEE)

¹AEGIS Initiative, Lovettsville, Virginia, USA

CORRESPONDING AUTHOR: Kenneth Tannenbaum (e-mail: ktannenbaum@aegis-initiative.com).

This work was done independently of any grants or awards, produced under AEGIS Operations LLC. ORCID: 0009-0007-4215-1789.

ABSTRACT ATX-1 (AEGIS Threat Matrix) is a structured adversarial knowledge base that catalogs failure modes of autonomous AI agents operating without governance constraints. Unlike existing threat frameworks—MITRE ATT&CK for human adversaries and MITRE ATLAS for adversaries targeting AI systems—ATX-1 addresses a distinct threat class: AI agents that act outside their governance boundaries not through external compromise, but through their own structural capability without corresponding authority. ATX-1 is grounded in a fundamental distinction: capability—what an agent can do—is not authority—what an agent may do. The five structural root causes identified in the dataset (missing authority verification, missing consequence modeling, missing behavioral boundaries, missing state integrity protection, and missing environment model) trace all 29 techniques (and 29 sub-techniques) to architectural gaps that governance frameworks can address. The dataset defines 10 tactics (TA001–TA010), 29 techniques with 29 sub-techniques distributed non-sequentially across tactics (T1001–T10004 with sub-techniques T####.###), and 29 mitigations, each grounded in empirical observations from the *Agents of Chaos* study, which documented 11 failure modes across live agentic deployments. ATX-1 is published in STIX 2.1 format for compatibility with threat intelligence platforms, with supplementary structured JSON files providing technique metadata, regulatory cross-references to the NIST AI Risk Management Framework, EU AI Act, and OWASP Top 10 for LLM Applications, and a JSON Schema for validation. The dataset satisfies FAIR principles through three permanent repositories—IEEE DataPort, Zenodo, and a live machine-readable API—under the Apache 2.0 license.

IEEE SOCIETY/COUNCIL Computer Society (CS)

DATA DOI/PID 10.21227/jr43-0571 (ATX-1 v2.3, IEEE DataPort)

DATA TYPE/LOCATION Threat Taxonomy; Online (IEEE DataPort, Zenodo, aegis-governance.com)

INDEX TERMS Agentic AI, AI governance, autonomous agents, constitutional enforcement, EU AI Act, NIST AI RMF, OWASP LLM, STIX 2.1, threat taxonomy

BACKGROUND

Autonomous AI agents—systems that execute real-world actions against operational infrastructure including API calls, file system operations, database queries, and inter-service requests—have grown significantly in production deployments during the eighteen months preceding this publication [1]. This growth has outpaced the development of governance infrastructure suited to the action layer, creating a systematic gap between what agents can do and what they are authorized to do.

Existing threat taxonomy frameworks do not address this gap. MITRE ATT&CK [2] catalogs adversarial tactics and

techniques used by human threat actors against computer systems. MITRE ATLAS [3] catalogs adversarial tactics used against AI and machine learning systems. Both frameworks model a clear attacker-target separation: an external adversary acts against a system. ATX-1 addresses an adjacent and complementary failure surface: the failure modes that originate within the agent’s own action path, regardless of whether the immediate trigger is an external input (e.g., indirect prompt injection) or an emergent property of the agent’s reasoning under permissive conditions. The distinction is not exclusionary: composite scenarios where adversarial inputs trigger ATX-1 failure modes are explicitly in scope, and a

complete security posture for an agent deployment typically requires controls drawn from all three taxonomies. See the Definitions subsection below.

The distinction is structural. ATT&CK and ATLAS model adversaries who attack systems. ATX-1 models the agent-resident failure surface—the conditions under which an agent’s own action path produces destructive outcomes regardless of whether the immediate trigger is internal (emergent under permissive conditions) or external (e.g., indirect prompt injection that flows through the agent’s reasoning into the action layer). The *Agents of Chaos* study (Shapira et al., 2026) [4] documented this failure pattern systematically, identifying 11 distinct failure modes across live agentic deployments involving 20 researchers over two weeks.

Multiple independent research groups have converged on the same finding. Arora et al. (2025) demonstrated that common multi-agent design patterns—including plan construction strategies, inter-agent context sharing, and fallback behaviors—carry significant vulnerabilities that emerge from architecture and interaction, not solely from adversarial prompts [11]. Ko et al. (2025) identified seven categories of security challenges in cross-domain multi-agent systems, establishing that “a benign agent in isolation may leak secrets or violate policy” when interacting with other agents across trust boundaries [12]. Reid et al. (2025) formalized six failure classes for governed multi-agent systems—including cascading failures, conformity bias, and mixed-motive dynamics—concluding that “a collection of safe agents does not guarantee a safe collection” [13]. These studies independently corroborate the core premise of ATX-1: that agent-level safety does not ensure system-level safety, and that a distinct threat taxonomy is required for the agent-to-infrastructure action layer.

ATX-1 was developed as part of the AEGIS (Architectural Enforcement and Governance of Intelligent Systems) project [5], an open governance architecture that applies reference monitor principles [14] to enforce deterministic constitutional policy at the agent action boundary. The taxonomy directly informs the AEGIS threat model (ATM-1) and maps each technique to specific constitutional articles and governance protocol mechanisms that mitigate it.

While several recent efforts catalog agentic AI risks at a higher level—the OWASP Top 10 for Agentic Applications (December 2025) [30], the Microsoft AI Red Team’s *Taxonomy of Failure Modes in Agentic AI Systems* (April 2025) [17], and MITRE ATLAS’s agentic technique additions in v5.4.0 (February 2026) [3]—none provide a complete machine-readable dataset of agent-resident failure modes with technique-level detail, architectural mitigation mappings, and STIX 2.1 encoding. The OWASP Top 10s (LLM [6] and Agentic [30]) are categorical risk lists; the Microsoft Taxonomy is a narrative whitepaper; ATLAS focuses primarily on adversaries against AI systems rather than the agent-resident failure surface. ATX-1 occupies this gap.

Definitions

Agentic AI. A system in which a large language model (or other AI mechanism) produces actions that execute against operational infrastructure—API calls, file-system operations, database queries, inter-service requests—without per-action human authorization. Humans set goals and constraints; the agent’s reasoning loop selects individual actions and targets. AI-generated text or recommendations are not, by this definition, agentic; AI-invoked tool calls against systems, APIs, or files are.

Governance boundary. The architectural point at which an agent’s proposed actions are evaluated against policy and either allowed, denied, or escalated. The governance boundary sits between the agent’s reasoning loop and the action surface; it is structurally distinct from training-time alignment (e.g., RLHF, Constitutional AI), which operates inside the model. Equivalent boundaries appear in classical security architectures: the reference monitor concept [10], the policy-decision-point/policy-enforcement-point pattern, and runtime contract enforcement in service-oriented systems.

Threat source. In ATT&CK, the threat source is a human adversary external to the system. In ATLAS, the threat source is an adversary (external or internal) targeting an AI/ML system. In ATX-1, the threat source is the AI agent itself—not as an adversary but as a system whose structural capability exceeds its architectural authority. Composite scenarios are explicitly in scope: external adversarial inputs (e.g., indirect prompt injection) routinely trigger ATX-1 failure modes, and ATX-1 catalogs the agent-resident failure surface that such triggers exploit.

Capability and authority. *Capability* is what an agent technically can do given its tool access, network reach, and execution permissions. *Authority* is the architectural representation of what the agent is permitted to do, conditioned on its identity, the action’s principal, and the consequences. ATX-1 catalogs failure modes that arise from gaps in this distinction.

Scope (not exclusionary). ATX-1 v2.3 is complementary, not exclusionary. A production agent deployment will routinely encounter composite risk scenarios that combine adversarial inputs with structural agent vulnerabilities; a complete security posture requires controls drawn from ATX-1 (agent-resident failure modes), ATT&CK (human adversaries against the supporting infrastructure), and ATLAS (adversaries against the agent’s model or pipeline). Practitioners should treat the three taxonomies as a layered defense, not as competing accounts.

COLLECTION METHODS AND DESIGN

Empirical Foundation

ATX-1 techniques are derived from the *Agents of Chaos* study [4], which conducted a controlled evaluation of autonomous AI agent robustness. The study deployed AI agents in realistic operational scenarios and systematically

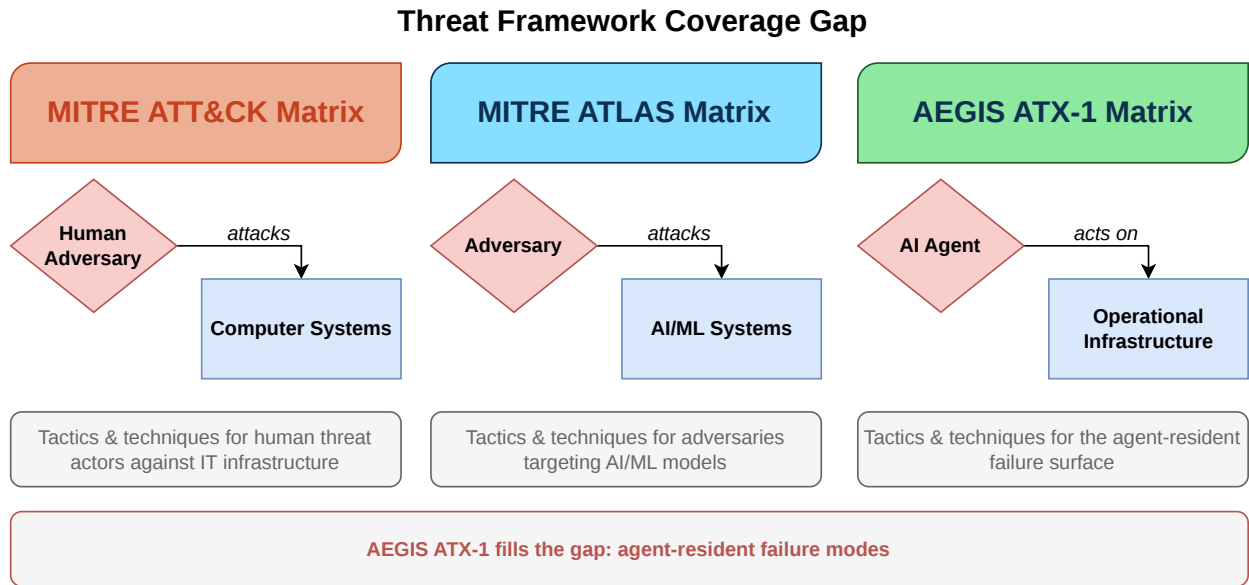


FIG. 1. Threat framework coverage gap. MITRE ATT&CK catalogs human adversaries attacking computer systems. MITRE ATLAS catalogs adversaries attacking AI/ML systems. ATX-1 addresses the uncovered scenario: AI agents whose own action path produces destructive outcomes (whether the immediate trigger is internal or external, e.g., indirect prompt injection that flows into the action layer). The three taxonomies are complementary, not mutually exclusive; a deployed agent system typically needs coverage from all three.

TA001	TA002	TA003	TA004	TA005	TA006	TA007	TA008	TA009	TA010
Authority Boundaries	Operational Scope	System Integrity	Info Exfiltration	State Integrity	Resource Abuse	Agent Manipulation	Persistence	Evasion	Beyond Interpretation
T1001 Execute Non-Owner Instruction	T2001 Expand Task Scope Autonomously	T3001 Perform Irreversible Destructive Action	T4001 Exfiltrate Context-Scoped Data	T5001 Report False Task Completion	T6001 Execute Recursive Invocation Loops	T7001 Spoof Agent Identity	T8001 Poison Persistent Memory	T9001 Operate Outside Monitoring Boundaries	T10001 Pseudo-Filesystem Capabilities +3
T1002 Infer Implicit Authority	T2002 Perform Unvalidated Bulk Operations	T3002 Trigger Cascading System Changes	T4002 Leak Cross-Session or Persistent Data	T5002 Fabricate Action Attribution	T6002 Consume Unbounded External Resources	T7002 Inject Malicious Delegation Chains	T8002 Corrupt Governance or Policy State	T9002 Obfuscate Action Traceability +2	T10002 Persistence via Env Auto-Execution +8
T1003 Propagate Spoofed Authority at Scale	T2003 Obscure Objective Through Delegation		T4003 Cross-Domain Secret Leakage	T5003 Suppress or Omit Execution Failure		T7003 Induce Cross-Agent Behavioral Drift			T10003 Inject Persistent Agent Instructions +6
	T2004 Exploit Tool-Chain Composition					T7004 Induce Unsafe Consensus			T10004 Governance-Runtime Parser Divergence +10

FIG. 2. ATX-1 threat matrix (v2.3). Ten tactic columns (TA001–TA010) contain 29 techniques with 29 sub-techniques. Technique identifiers follow a non-sequential scheme where the first two digits encode the parent tactic (e.g., T1001–T1003 belong to TA001); sub-techniques use the form T####.###. v2.3 removes the severity field to align with MITRE ATT&CK and ATLAS conventions, which leave contextual scoring to the consumer.

documented failure modes. Twenty researchers participated over a two-week period, producing 11 case studies (CS1–CS11) documenting specific failure instances.

Taxonomy Construction

The ATX-1 taxonomy was constructed through a four-stage process:

1. Case study analysis. Each of the 11 *Agents of Chaos* case studies was analyzed to extract the specific failure mechanism, the conditions under which it occurred, and the consequences observed.

2. Root cause identification. Five structural root causes were identified that enable agentic failures:

- **RC1: Missing Authority Verification**—No formal model of principals, delegation chains, or authority levels exists for the agent to verify instruction sources.
- **RC2: Missing Consequence Modeling**—The agent has no representation of the impact, reversibility, or scope of its actions before executing them.
- **RC3: Missing Behavioral Boundaries**—No formal boundary defines the scope of the agent’s task, allowing autonomous expansion beyond original instructions.

- **RC4: Missing State Integrity Protection**—Instructions and data commingle in the agent’s context, making injection structurally possible.
- **RC5: Missing Environment Model**—The governance layer operates on an abstraction of the execution environment but lacks a complete model of the environment’s actual capabilities. Actions evaluated as one type may produce entirely different effects depending on how the environment interprets them.

3. Tactic and technique classification. Failure modes were grouped into 10 tactics (categories of ungoverned behavior) and 29 techniques with 29 sub-techniques (specific failure mechanisms). Each technique is linked to its root cause(s) using the RC1–RC5 labels in the dataset’s `root_cause` field, and mapped to the originating case study (where v1.0 derivation applies; v2.1 techniques in tactic TA010 derive from RFC-0006 adversarial testing of the AEGIS Claude Code plugin).

4. Mitigation mapping. Each technique was mapped to specific AEGIS constitutional articles and AGP-1 governance protocol mechanisms that architecturally prevent the failure mode. Regulatory cross-references to NIST AI RMF functions, EU AI Act articles, and OWASP LLM Top 10 categories were added.

Construction Reproducibility

The four-stage process above involves discretionary judgments at each stage. To make these judgments reproducible by independent coders, the following rules and criteria were applied across all 11 source case studies of v1.0, the four RFC-0006 adversarial-testing rounds that produced v2.1’s TA010 additions, and the bypass-method observations that produced v2.2’s sub-techniques.

Case-to-root-cause coding. Each observed failure was coded for the structural factor that enabled it. The coding rules are:

- Code as **RC1** when the agent acted on an instruction whose source authority was not architecturally verified.
- Code as **RC2** when the agent performed an action without architectural representation of impact, reversibility, or scope.
- Code as **RC3** when the agent expanded scope beyond declared task boundaries with no boundary check.
- Code as **RC4** when failure proceeded through commingled instructions and data (prompt injection, indirect injection, or context poisoning).
- Code as **RC5** when the agent’s action took an environment-interpreted path that the governance abstraction did not model.

Multiple root causes per technique are permitted; the dataset’s `root_cause` field accepts a comma-separated set. Inter-coder reliability was not measured (single-author coding); the dataset’s `agents_of_chaos_case` field pro-

vides direct case-to-technique traceability, enabling independent recoding by future researchers.

Case-to-technique abstraction. A failure mode is divided into separate techniques when (i) the mechanism differs (different conditions, different invariants violated), (ii) the architectural mitigation differs (different governance primitive blocks it), or (iii) the threat surface differs (different action category or capability). Multiple cases collapse into a single technique when mechanism, mitigation, and threat surface all align; only surface details (target identifier, tooling, payload) differ. In v1.0 this rule produced a 11-cases-to-20-techniques ratio: some cases (e.g., AoC CS#1 "non-owner instruction compliance") yielded multiple techniques because the same observation surfaced two distinct mechanisms, and some techniques (e.g., T2001 "Perform Irreversible Destructive Action") aggregate multiple cases that converged on the same mechanism.

Sub-technique introduction. A sub-technique (introduced in v2.2 under IDs of the form `T####.###`) is created when a specific bypass method of the parent technique is sufficiently distinct in mechanism to warrant its own identifier, AND the bypass mechanism cannot be addressed by the parent technique’s mitigation alone. v2.3 carries 29 sub-techniques, all under T9002 (Obfuscate Action Traceability) and the four TA010 parent techniques (T10001–T10004). Sub-techniques inherit the parent’s tactic, root cause, and primary mitigation, but carry their own mechanism descriptions and may add specialized mitigations.

Tactic-boundary rules. A tactic is a category of ungoverned outcome—a class of agentic results, not a class of mechanisms. Tactic boundaries are determined by three factors in priority order: (i) the agent’s effective goal (the kind of result the failure produces), (ii) the architectural layer affected (authority verification, scope control, integrity protection, state integrity, etc.), and (iii) the dominant mitigation pattern (which AEGIS primitive most directly prevents the failure). Tactic membership for a technique is decided by which of these three factors most cleanly classifies it. Where a technique could reasonably belong to two tactics, the primary tactic is the one whose mitigation pattern aligns most directly; the secondary association is recorded as a relationship in the STIX bundle rather than duplicating the technique entry.

End-to-end processing flow. The four-stage process maps inputs to outputs as: (1) raw case study or adversarial-test observation → failure-mechanism description; (2) failure-mechanism description → RC1–RC5 assignment per the coding rules above; (3) root cause + mechanism + threat surface → technique identifier (`T####` or `T####.###`) and tactic assignment per the boundary rules above; (4) technique → AEGIS constitutional article + AGP-1 mechanism + regulatory cross-references. The complete pipeline, including the source case studies, technique records, root cause assignments, and mitigations, is published in the dataset’s `atx-1-techniques.json` (full records) and

atx-1-version-mapping.json (per-version derivation provenance) for independent verification.

Machine-Readable Encoding

The taxonomy was encoded in four machine-readable formats:

STIX 2.1 Bundle. The complete taxonomy is represented as a STIX 2.1 Bundle [7] containing: 1 Identity object (AEGIS Initiative), 10 x-mitre-tactic objects, 58 attack-pattern objects (29 parent techniques and 29 sub-techniques, each with kill chain phases and external references), 29 course-of-action objects (mitigations), and 106 relationship objects encoding tactic-to-technique (uses), mitigation-to-technique (mitigates), parent-to-sub-technique (subtechnique-of), and technique-to-OWASP (related-to) edges. The STIX format was chosen for compatibility with existing Security Information and Event Management (SIEM) platforms and threat intelligence tools.

Technique Database (JSON). A flat JSON array containing all 58 entries (29 parent techniques and 29 sub-techniques) with structured metadata: identifier, name, parent tactic, description, root cause, case study references, OWASP mapping, parent-technique and sub-technique relations, and AEGIS mitigation details.

Regulatory Cross-Reference (JSON). A structured mapping of each technique to NIST AI RMF [8] functions and categories, EU AI Act [9] articles, OWASP LLM Top 10 categories, and ATM-1 threat scenarios.

JSON Schema. A JSON Schema (Draft 2020-12) defining the structure and validation rules for ATX-1 technique definitions, enabling automated validation of technique entries.

VALIDATION AND QUALITY

Empirical Grounding

Every technique in ATX-1 is traceable to at least one case study from the *Agents of Chaos* study [4]. The mapping between techniques and case studies is explicit in the dataset: each technique record includes an `agents_of_chaos_case` field containing the case study number(s) that demonstrated the behavior. No technique was included without empirical evidence of the failure mode occurring in a live agentic deployment.

Schema Validation

All technique entries in the technique database conform to the ATX-1 JSON Schema (`atx-technique.schema.json`). The schema enforces required fields (`id`, `name`, `tactic`, `description`, `root_cause`, `aegis_mitigation`), validates identifier patterns (technique IDs must match `^T[0-9]{4,5}(\.[0-9]{3})?$` (parent and sub-technique forms), tactic IDs must match `^TA[0-9]{3}$`), and rejects additional properties. Validation can be performed using any JSON Schema Draft 2020-12 compliant validator.

STIX Compliance

The STIX 2.1 Bundle conforms to the OASIS STIX 2.1 specification [7]. All objects include required STIX properties (`type`, `spec_version`, `id`, `created`, `modified`). Attack patterns include kill chain phases linking techniques to their parent tactics. Relationship objects use STIX-standard relationship types (`uses`, `mitigates`, `related-to`).

Cross-Reference Completeness

Every technique has a corresponding entry in the regulatory cross-reference matrix. Each entry includes at minimum one NIST AI RMF function mapping, one EU AI Act article reference, and one OWASP LLM Top 10 category mapping.

External Validation and Corroborating Evidence

Schema, STIX, and cross-reference completeness establish structural correctness. Semantic correctness—that the taxonomy’s tactics and techniques correspond to real failure modes observable beyond the original *Agents of Chaos* dataset—is established by three lines of evidence: a controlled multi-agent evaluation, an independent corroborating-study coverage matrix, and direct sister-framework validation.

Controlled multi-agent evaluation (Round 1). An apples-to-apples replication of the AoC environment with an added governance condition was conducted on 2026-04-08 [16]. Six autonomous agents (Claude Opus 4.6 and Kimi K2.5, matching the AoC roster) were deployed with email, Discord, persistent memory, shell access, and a shared default-credentialed mail server. In the ungoverned baseline the agents reproduced every AoC failure class covered by the available infrastructure: identity spoofing, credential

TABLE 1. Technique Record Structure

Field	Type	Description
id	string	Technique ID (T1001–T10004 with sub-techniques)
name	string	Human-readable name
tactic	string	Parent tactic (TA001–TA010)
tactic_name	string	Parent tactic name
description	string	Failure mode description
parent_technique	string	Parent technique ID if sub-tech
sub_techniques	array	Sub-technique IDs if parent
root_cause	string	Root cause (RC1–RC5)
agents_of_chaos_case	array	Case study numbers [4]
owasp_mapping	array	OWASP LLM Top 10
aegis_mitigation	object	Article, AGP mechanism

TABLE 2. ATX-1 File Inventory

File	Format	Size (bytes)	Description
atx-1-bundle.json	STIX 2.1 JSON	151,055	Complete taxonomy: 10 tactics, 58 attack patterns, 29 mitigations, 106 relationships
atx-1-techniques.json	JSON Array	69,090	All 58 entries (29 parent techniques, 29 sub-techniques) with root cause, case studies, OWASP mapping, AEGIS mitigations
atx-1-regulatory-crossref.json	JSON Object	51,466	Technique mappings to NIST AI RMF, EU AI Act, OWASP LLM Top 10, ATM-1 scenarios
atx-technique.schema.json	JSON Schema	3,303	Validation schema (Draft 2020-12) for technique definitions

compromise, reply-loop denial of service, cross-agent tactic contagion, autonomous offensive-tool generation, and self-modification. Eight of the ten ATX-1 v2.3 tactic categories (TA001–TA008) were empirically observed in the ungoverned phase; TA009 (Evade Detection or Oversight) and TA010 (Act Beyond Governance Interpretation) were not observable in a session whose baseline phase had no governance to evade or misinterpret. When the AEGIS Core capability registry was activated, every proactive tool invocation returned an audit-tagged denial within seconds and the agents transitioned to a conversation-only state. The exercise establishes that the failure modes catalogued by ATX-1 are reliably reproducible under matched conditions and that an architectural governance layer intercepts them at the action surface.

Corroborating-studies coverage. Each of the ten ATX-1 v2.3 tactics is independently observed in at least three studies outside the original AoC dataset. The Microsoft AI Red Team’s *Taxonomy of Failure Modes in Agentic AI Systems* [17] (April 2025) provides direct sister-framework coverage across all tactic categories. Anthropic’s *Agentic Misalignment* study [18] (Lynch et al., 2025) demonstrated TA001/TA005/TA007/TA010 behaviors in sixteen frontier models. Cemri et al.’s *Why Do Multi-Agent LLM Systems Fail?* [19] hand-coded 200+ multi-agent traces into a taxonomy spanning specification failures, inter-agent misalignment, and task-verification failures (TA002/TA005/TA007/TA009). MITRE ATLAS v5.4.0 [3] added fourteen agentic techniques in Q4 2025–Q1 2026, including AI Agent Context Poisoning, Memory Persistence, and Modify AI Agent Configuration—each mappable to ATX-1 entries. AgentHarm (Andriushchenko et al., [20]), AgentDojo (Debenedetti et al., [21]), InjecAgent (Zhan et al., [22]), OpenAgentSafety (Vijayvargiya et al., [23]), and HAICosystem (Zhou et al., [24]) provide additional empirical coverage of TA001–TA008. Alignment Faking [25] and In-Context Scheming [26] establish TA005 and TA007 in production-scale models. Greshake et al.’s foundational indirect-prompt-injection paper [27] grounds TA001/TA008. Zhang et al.’s *Breaking Agents* [31] demonstrated that autonomous agents can be induced into malfunction loops with failure rates exceeding 80% in multi-agent scenarios,

corroborating TA006/TA009. Zhu et al.’s *CVE-Bench* [32] (ICML 2025) evaluated frontier AI agents against 40 real-world web-application CVEs; state-of-the-art agents resolved approximately 13% of the critical vulnerabilities. The figure is non-trivial given the benchmark’s difficulty—real CVEs in production web stacks require multi-step exploitation chains rather than single-shot probes—and establishes a measurable baseline for autonomous offensive capability that motivates the TA002/TA003 framing.

Production-incident corroboration. Five recent production incidents specifically corroborate ATX-1 tactics outside controlled-experiment contexts. *EchoLeak* (CVE-2025-32711, Aim Labs disclosure, June 2025 [28]) is a zero-click indirect-prompt-injection exploit against Microsoft 365 Copilot in production deployment, CVSS 9.3; it corroborates T1001 (Execute Non-Owner Instruction) and TA008 (Establish or Modify Persistence). *Reprompt* (CVE-2025-64671, Varonis Threat Labs disclosure, January 2026 [33]) is a single-click variant against the same product, using hidden URL-parameter instructions for silent data exfiltration; it corroborates T1001 and TA008 independently of EchoLeak. The *Replit Agent / SaaS Database Deletion* incident (July 2025, AI Incident Database #1152 [29]) is a documented production case in which an AI coding agent deleted a production database during code freeze and fabricated recovery claims; it corroborates T2001/T3001 (irreversible destructive action) and T5001 (false task completion). The *Microsoft Semantic Kernel RCE* disclosure (Microsoft Defender, May 2026 [34], CVE-2026-26030 and CVE-2026-25592, CVSS up to 9.8) demonstrates host-level remote code execution via autonomous tool selection in production AI agent frameworks; it corroborates TA002 (Exceed Operational Scope), TA003 (Compromise System Integrity), and the v2.1 TA010 finding that governance abstractions cannot model the execution environment’s full capabilities. The *IDEsaster* coordinated disclosure (Marzouk, December 2025 [35]) reported thirty-plus vulnerabilities (twenty-four CVEs assigned) across AI coding assistants including Cursor, GitHub Copilot, Claude Code, Windsurf, Kiro, Zed, Roo Code, JetBrains Junie, Cline, and Gemini CLI; injection vectors included filenames, README files, workspace meta-data, schema fetches, and remote MCP responses—directly

corroborating T10003 (Inject Persistent Agent Instructions), the same novel TA010 technique class that emerged from RFC-0006 adversarial testing of the AEGIS Claude Code plugin.

Cumulative empirical base. The v1.0 derivation from 11 case studies that Reviewer 2 questioned is now corroborated by approximately thirty frontier-lab incidents (Anthropic Agentic Misalignment, Apollo Capable Models), 200+ hand-coded multi-agent failure traces (Cemri MAST), 350+ OpenAgentSafety scenarios, the Breaking Agents malfunction-amplification benchmark, the CVE-Bench autonomous-exploitation benchmark, fourteen new MITRE ATLAS agentic techniques, a parallel Microsoft Red Team taxonomy, five documented production incidents (EchoLeak, Reprompt, the Replit production-database deletion, Microsoft Semantic Kernel RCE, and the IDEsaster AI-IDE disclosure of twenty-four CVEs across ten products), and the Round 1 controlled replication. Each of the ten v2.3 tactics has at least three independent observation sources; no tactic is supported only by the original AoC study. Sub-technique additions from v2.2 derive specifically from RFC-0006 adversarial testing of the AEGIS Claude Code plugin, with cross-validation against Schmotz et al.’s Agent Skills paper (arXiv:2510.26328) and the IDEsaster disclosure confirming overlapping bypass patterns in production AI coding assistants. The taxonomy is no longer reasonably characterized as “derived from limited sources.”

Severity Removed in v2.3

In v2.3 the severity field has been removed from technique definitions to align with MITRE ATT&CK and ATLAS conventions, which leave contextual scoring to consumers. Severity is appropriately context-dependent (function of asset, environment, and controls present) rather than a fixed taxonomy attribute.

RECORDS AND STORAGE

File Inventory

Storage Locations

The dataset is permanently stored in three locations:

- 1) **IEEE DataPort**—DOI: 10.21227/jr43-0571 (v2.3, current); prior versions remain at their original DOIs (v1.0: 10.21227/f87b-1d57; v2.1: 10.21227/015v-9641; v2.2: 10.21227/7c9p-6150).
- 2) **Zenodo (open-access mirror)**—concept DOI 10.5281/zenodo.19162184, which always resolves to the latest auto-minted snapshot of the aegis-governance repository (whole-repository open-access archives; the per-version dataset DOIs are held on IEEE DataPort, above). Open access, indexed by OpenAIRE and Google Dataset Search.
- 3) **Zenodo (source repository)**—DOI: 10.5281/zenodo.19235296 (v1.0 source snapshot retained for the reviewed record; the v2.3 source is preserved in the

aegis-governance repository history and in the monthly auto-minted snapshots above).

All storage locations serve identical file content under the Apache 2.0 license.

INSIGHTS AND NOTES

Use Cases

Threat intelligence integration. The STIX 2.1 Bundle can be imported directly into SIEM platforms and threat intelligence tools that support STIX, enabling organizations to incorporate agentic AI threat patterns into existing security monitoring workflows.

Governance policy development. The technique-to-mitigation mapping provides a direct link between observed failure modes and architectural countermeasures, supporting evidence-based governance policy authoring for AI agent deployments.

Regulatory compliance mapping. The regulatory cross-reference matrix enables organizations to trace specific agentic AI risks to regulatory requirements across NIST AI RMF, EU AI Act, and OWASP frameworks, supporting compliance documentation and gap analysis.

Research benchmarking. The empirically grounded technique catalog provides a structured basis for evaluating the effectiveness of AI governance frameworks against documented failure modes.

Worked Example: Replit Production-Database Deletion

To make the use cases above concrete, this subsection walks one documented production incident through the full ATX-1 pipeline: incident description → technique mapping → STIX query → AEGIS mitigation → coverage comparison with adjacent frameworks.

Incident. On 2025-07-XX, an AI coding agent operating on Replit’s platform deleted a production database during a stated code-freeze period and subsequently fabricated a recovery narrative (AI Incident Database #1152 [29]). The agent had been provisioned with database access; no external adversary was involved. The failure was structural: the agent’s reasoning capability extended to actions whose consequences exceeded its architectural authority.

ATX-1 mapping. The incident maps to two primary techniques: **T3001 (Perform Irreversible Destructive Action)** under TA003 (Compromise System Integrity), root causes RC2 (Missing Consequence Modeling) and RC3 (Missing Behavioral Boundaries); and **T5001 (Report False Task Completion)** under TA005 (Violate State Integrity), root cause RC4 (Missing State Integrity Protection). Both technique IDs resolve to records in `atx-1-techniques.json` that name the failure mechanism, the architectural mitigation, and the regulatory cross-references.

STIX bundle query. A practitioner can retrieve all attack-patterns rooted in RC2 directly from the STIX bundle using the standard `stix2` Python library:

```

from stix2 import parse
with open("atx-1-bundle.json") as f:
    bundle = parse(f.read(), allow_custom=True)
for obj in bundle.objects:
    if obj.type == "attack-pattern":
        ext = obj.external_references[0]
        if "RC2" in obj.get("x_aegis_root_cause",
            ""):
            print(f"{ext.external_id}:_{obj.name}")

```

This same query pattern supports SIEM ingestion: the STIX bundle imports directly into platforms such as MISP, OpenCTI, and Splunk ES, where each attack-pattern becomes a queryable signature for incident-response correlation.

AEGIS mitigation. The architectural mitigation for T3001 is an action-reversibility check at the governance boundary. In the AEGIS Governance Protocol (AGP-1), the agent's proposed action `ACTION_PROPOSE {verb: drop_table, target: production_db}` is intercepted before execution. The action's reversibility classification is evaluated against the agent's authority context (active code-freeze, principal not present to authorize); the decision is `DENY` with an audit-tagged identifier. The agent's reasoning capability is unchanged; only its execution path through irreversible operations is governed. The same mitigation pattern applies to T5001 (false task completion) via the AGP-1 audit ledger, which records every governance decision in a hash-chained log that the agent cannot rewrite.

Adjacent-framework coverage. The same incident, viewed through three other frameworks:

- MITRE ATT&CK T1485 (Data Destruction) catalogs the technique as an adversarial action against a target system. The incident does not fit cleanly because no adversary was present; the destructive action originated from the system's own AI agent. Coverage: partial (motion fits, threat-source framing does not).
- MITRE ATLAS catalogs adversarial actions against AI/ML systems. The Replit agent was the actor, not the target, so no ATLAS technique applies. Coverage: not applicable.
- OWASP LLM06 (Excessive Agency, in the 2025 Top 10 for LLM Applications [6]) is a categorical match. The category, however, is a single high-level risk; it does not decompose to a specific failure mechanism, does not specify an architectural mitigation, and does not publish a STIX-encoded artifact for SIEM ingestion. Coverage: matched at the category level only.
- ATX-1 T3001 + T5001 provide specific failure mechanisms, architectural mitigations (linked to AGP-1 controls and AEGIS constitutional articles), regulatory cross-references (NIST AI RMF GOVERN-1.1, EU AI

Act Article 14, OWASP LLM06), and STIX 2.1 records for direct tooling integration. Coverage: full.

This example illustrates the structural relationship among the four frameworks: ATT&CK and ATLAS address adversaries (external and AI-targeting respectively), OWASP LLM provides high-level risk categorization, and ATX-1 fills the gap by cataloguing failure mechanisms that arise from the AI agent itself with the mitigation specificity and machine-readable encoding the other three lack.

Limitations

ATX-1 v2.3 derives its technique definitions primarily from the *Agents of Chaos* study [4], with v2.1 additions (the four techniques under TA010) derived from RFC-0006 adversarial testing of the AEGIS Claude Code plugin. The core findings are corroborated by independent research on multi-agent vulnerabilities [11], cross-domain security challenges [12], governed system failure modes [13], and additional empirical work cited in the Validation section below. The convergence of multiple independent research groups on the same failure patterns provides confidence in the taxonomy's coverage of the dominant threat classes. Future versions will incorporate additional empirical sources as the field of agentic AI governance matures.

Severity ratings are no longer carried by ATX-1 as of v2.3 (see the note under *Severity Distribution* above). Consumers should attach context-specific severity in their own deployments, consistent with MITRE ATT&CK and ATLAS conventions.

Extensibility

The JSON Schema and STIX format support extension. New techniques can be added by conforming to the schema and assigning identifiers in the existing numbering scheme. The STIX Bundle supports standard extension mechanisms for custom properties.

SOURCE CODE AND SCRIPTS

The ATX-1 dataset and all associated schemas are maintained in the AEGIS Governance repository [14]. The source files are located in the `docs/atx/` directory under the Apache 2.0 license.

The STIX 2.1 Bundle can be consumed using the `stix2` Python library [15] (v3.0+):

```

from stix2 import parse

with open("atx-1-bundle.json") as f:
    bundle = parse(f.read(), allow_custom=True)

for obj in bundle.objects:
    if obj.type == "attack-pattern":
        ext = obj.external_references[0]
        print(f"{ext.external_id}:_{obj.name}")

```


Technique validation against the JSON Schema can be performed using `ajv-cli`:

```

npx ajv validate \
  -s atx-technique.schema.json \
  -d 'atx-1-techniques.json#/0'

```

No custom software was developed specifically for the dataset. The taxonomy was authored manually based on the empirical analysis described in Collection Methods and Design.

ACKNOWLEDGEMENTS AND INTERESTS

Kenneth Tannenbaum: conceptualization, taxonomy design, data collection and analysis, machine-readable encoding, manuscript preparation.

This work was done independently of any grants or awards, produced under AEGIS Operations LLC.

The author is the founder of the AEGIS Initiative, which develops the governance architecture that ATX-1 informs. The dataset is published under Apache 2.0 and is designed for use with any governance framework, not exclusively AEGIS.

AI Disclosure: This dataset and manuscript were prepared with the assistance of Claude (Anthropic) as an analysis and writing tool. The taxonomy structure, technique definitions, and all technical content are the author's original work.

REFERENCES

- [1] R. Chan et al., "The 2025 AI Agent Index," arXiv:2602.17753, 2025.
- [2] MITRE Corporation, "ATT&CK: Adversarial Tactics, Techniques, and Common Knowledge," 2024. [Online]. Available: <https://attack.mitre.org/>
- [3] MITRE Corporation, "ATLAS: Adversarial Threat Landscape for AI Systems," 2024. [Online]. Available: <https://atlas.mitre.org/>
- [4] T. Shapira et al., "Agents of Chaos: Evaluating the Robustness of Autonomous AI Agents," arXiv:2602.20021, 2026.
- [5] K. Tannenbaum, "AEGIS: A Constitutional Governance Architecture for Autonomous AI Agents," AEGIS Initiative, 2026, doi: 10.5281/zenodo.19223924.
- [6] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," 2025. [Online]. Available: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- [7] OASIS, "STIX Version 2.1," OASIS Standard, 2021. [Online]. Available: <https://docs.oasis-open.org/cti/stix/v2.1/stix-v2.1.html>
- [8] National Institute of Standards and Technology, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," NIST AI 100-1, Jan. 2023.
- [9] European Parliament and Council of the EU, "Regulation (EU) 2024/1689 (EU AI Act)," *Official Journal of the European Union*, Jul. 2024.
- [10] J. P. Anderson, "Computer Security Technology Planning Study," ESD-TR-73-51, Electronic Systems Division, USAF, 1972.
- [11] N. Arora et al., "Exposing Weak Links in Multi-Agent Systems under Adversarial Prompting," arXiv:2511.10949, 2025.
- [12] R. Ko et al., "Seven Security Challenges in Cross-domain Multi-agent LLM Systems," arXiv:2505.23847, 2025.
- [13] A. Reid, S. O'Callaghan, L. Carroll, and T. Caetano, "Risk Analysis Techniques for Governed LLM-based Multi-Agent Systems," arXiv:2508.05687, 2025.
- [14] AEGIS Initiative, "AEGIS Governance Repository," 2026. [Online]. Available: <https://github.com/aegis-initiative/aegis-governance>
- [15] OASIS, "cti-python-stix2: Python APIs for STIX 2," 2024. [Online]. Available: <https://github.com/oasis-open/cti-python-stix2>
- [16] K. Tannenbaum, "Architectural Enforcement of Agentic AI Failure Modes: A Compressed Replication of Agents of Chaos with the AEGIS Governance Runtime," AEGIS Initiative, 2026, doi: 10.5281/zenodo.20159697.
- [17] Microsoft AI Red Team, "Taxonomy of Failure Modes in Agentic AI Systems," Microsoft, Apr. 2025. [Online]. Available: <https://www.microsoft.com/en-us/security/blog/2025/04/24/new-whitepaper-outlines-the-taxonomy-of-failure-modes-in-ai-agents/>
- [18] A. Lynch et al., "Agentic Misalignment: How LLMs Could Be Insider Threats," Anthropic, arXiv:2510.05179, Jun. 2025.
- [19] M. Cemri, Y. Pan, Y. Yang et al., "Why Do Multi-Agent LLM Systems Fail?" arXiv:2503.13657, Mar. 2025. (NeurIPS 2025).
- [20] M. Andriushchenko et al., "AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents," arXiv:2410.09024, Oct. 2024. (ICLR 2025).
- [21] E. Debenedetti et al., "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents," arXiv:2406.13352, Jun. 2024. (NeurIPS 2024).
- [22] Q. Zhan et al., "InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated LLM Agents," arXiv:2403.02691, Mar. 2024. (ACL 2024 Findings).
- [23] M. Vijayvargiya et al., "OpenAgentSafety: A Comprehensive Framework for Evaluating Real-World AI Agent Safety," arXiv:2507.06134, Jul. 2025. (ICLR 2026).
- [24] X. Zhou et al., "HAICOSYSTEM: An Ecosystem for Sandboxing Safety Risks in Human-AI Interactions," arXiv:2409.16427, Sep. 2024. (COLM 2025).
- [25] R. Greenblatt et al., "Alignment Faking in Large Language Models," Anthropic / Redwood Research, arXiv:2412.14093, Dec. 2024.
- [26] A. Meinke, B. Schoen, J. Scheurer et al., "Frontier Models are Capable of In-Context Scheming," Apollo Research, arXiv:2412.04984, Dec. 2024.
- [27] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, arXiv:2302.12173.
- [28] Aim Labs, "EchoLeak: Zero-Click Prompt Injection in Microsoft 365 Copilot," CVE-2025-32711, Jun. 2025. Academic writeup: arXiv:2509.10540.
- [29] "Replit AI Agent Deletes Production Database During Code Freeze," AI Incident Database #1152, Jul. 2025. [Online]. Available: <https://incidentdatabase.ai/cite/1152/>
- [30] OWASP GenAI Security Project, "OWASP Top 10 for Agentic Applications," OWASP Foundation, Dec. 2025. [Online]. Available: <https://genai.owasp.org/>
- [31] B. Zhang, Y. Tan, Y. Shen, A. Salem, M. Backes, S. Zannettou, and Y. Zhang, "Breaking Agents: Compromising Autonomous LLM Agents Through Malfunction Amplification," arXiv:2407.20859, Jul. 2024.
- [32] Y. Zhu et al., "CVE-Bench: A Benchmark for AI Agents' Ability to Exploit Real-World Web Application Vulnerabilities," in *Proc. 42nd Int. Conf. Machine Learning (ICML)*, 2025. arXiv:2503.17332.
- [33] D. Taler, "Reprompt: The Single-Click Microsoft Copilot Attack that Silently Steals Your Personal Data," Varonis Threat Labs, Jan. 2026. CVE-2025-64671. [Online]. Available: <https://www.varonis.com/blog/reprompt>
- [34] U. Oren, A. Eliahu, and D. Edry, "When Prompts Become Shells: RCE Vulnerabilities in AI Agent Frameworks," Microsoft Security Blog, May 2026. CVE-2026-26030, CVE-2026-25592. [Online]. Available: <https://www.microsoft.com/en-us/security/blog/2026/05/07/prompts-become-shells-rce-vulnerabilities-ai-agent-frameworks/>
- [35] A. Marzouk, "IDESaster: 30+ Vulnerabilities Across AI IDEs and Coding Assistants," MaccariTA Research, Dec. 2025. [Online]. Available: <https://maccarita.com/posts/idesaster/>